LOCAL LIBRARY RESOURCE MANAGEMENT SYSTEM (LLRMS) BY Yash Yadav-24BCY12047

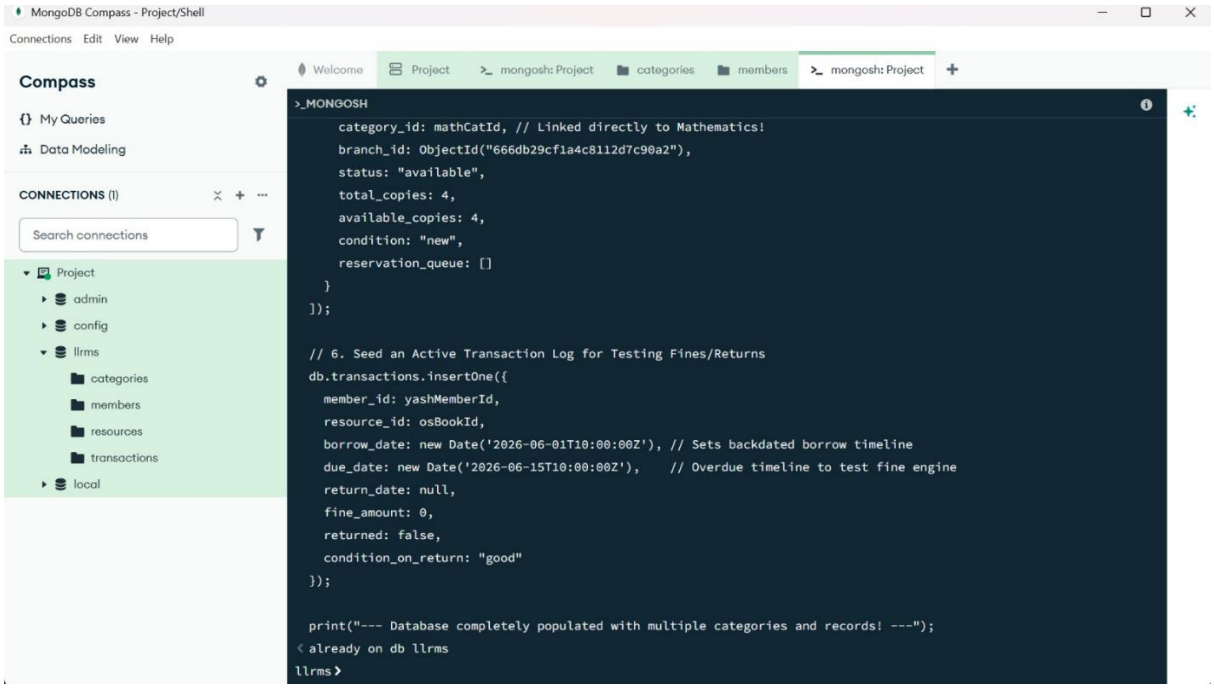
COMPLETE CORE SYSTEM VERIFICATION REPORT

📑 SECTION A: Database Architecture & Verification

This section verifies the underlying non-relational database structure inside MongoDB, showing multi-value documents, unique references, and relational integrity via ObjectIDs without traditional SQL constraints.

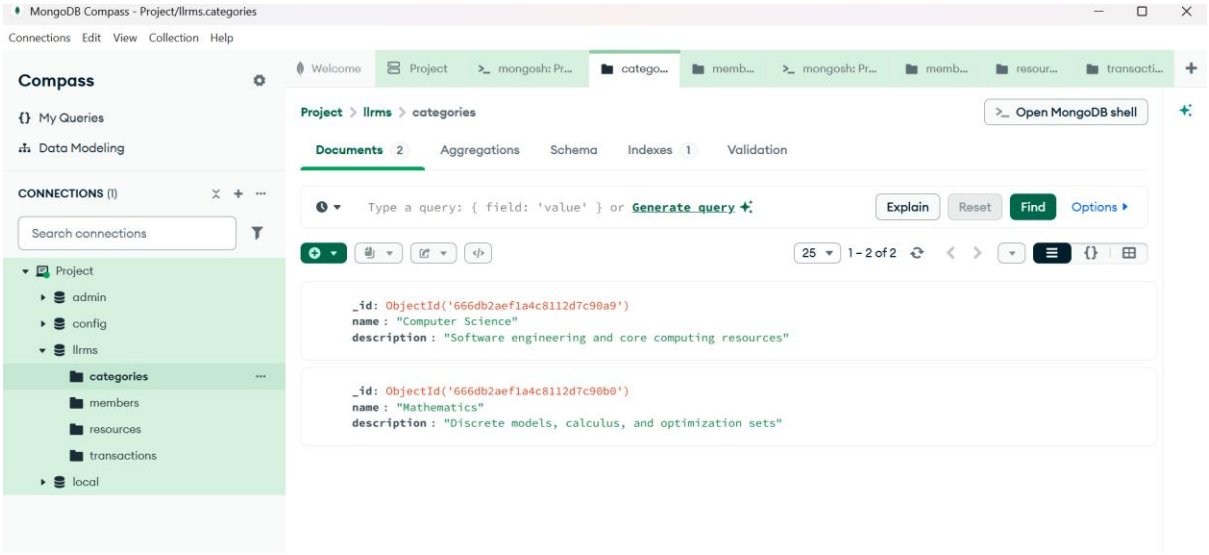
1. Database Population Script Execution

- **Description:** Verification of database seeding and initialization using native MongoDB shell scripting.
- **Workflow:** Open MongoDB Compass, click **>_ Open MongoDB shell** at the top right, paste the complete seeding array script, and press **Enter**.
- **Verification Marker:** Ensure the terminal window confirms execution with the line: --- Database completely populated with multiple categories and records! ---.

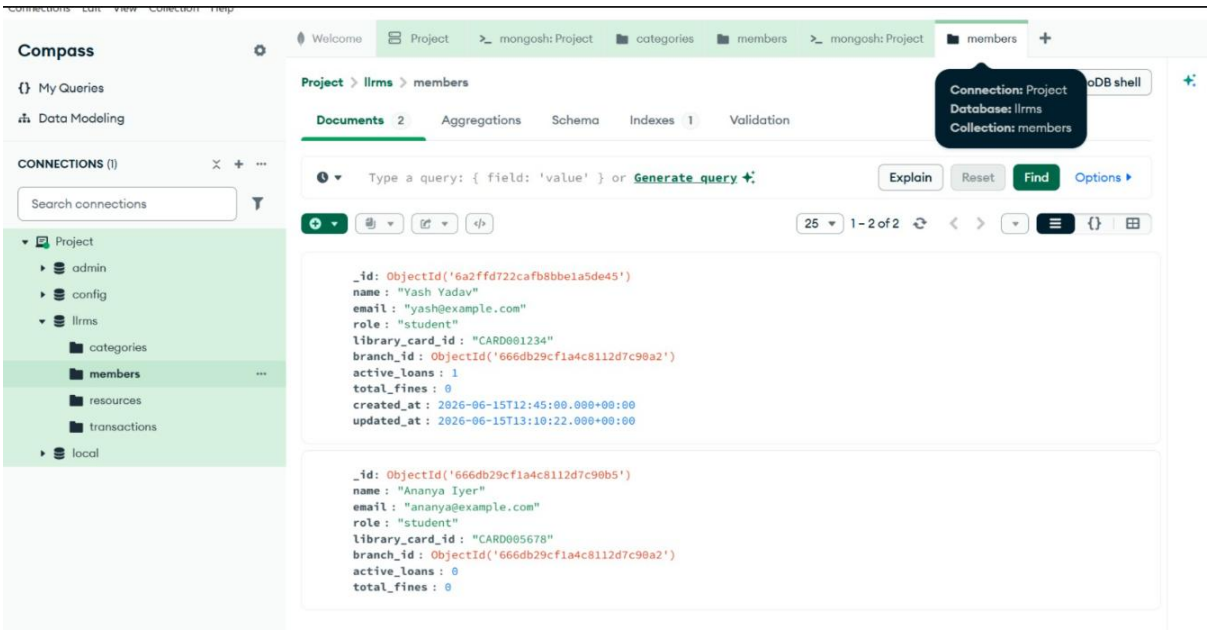


2. Categories Collection Matrix

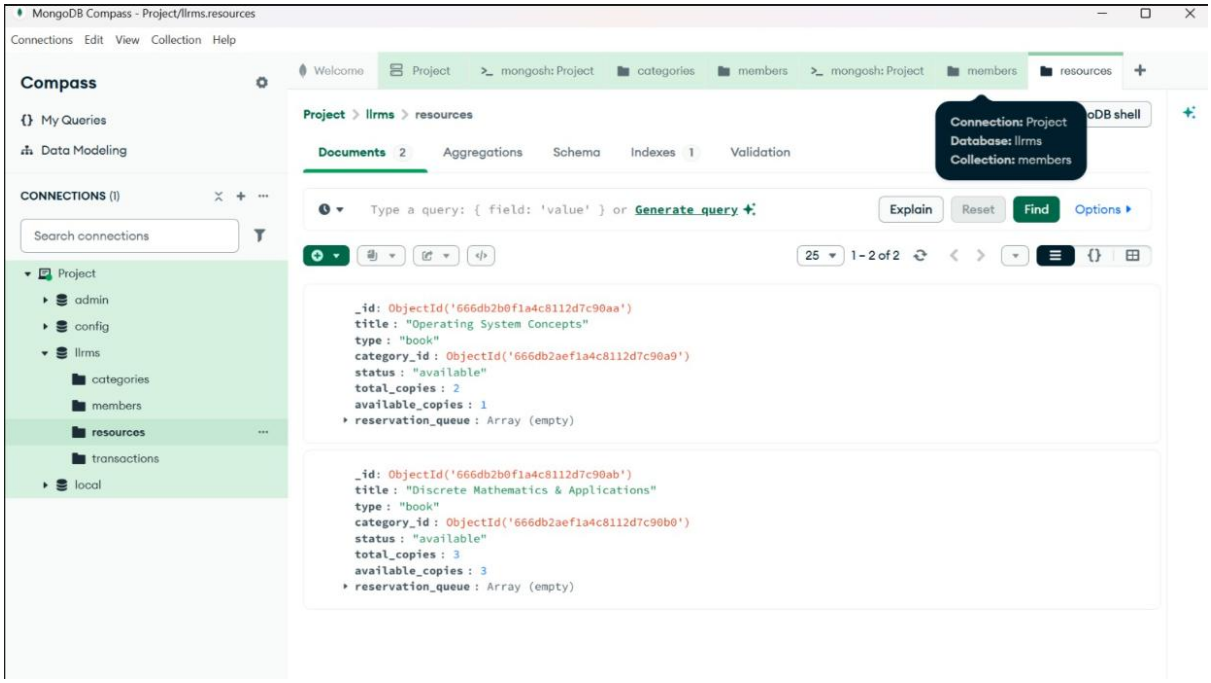
- **Description:** Proves successful data insertion and categorization mapping across multiple subjects (Computer Science and Mathematics).
- **Workflow:** Navigate to the llrms database tree in MongoDB Compass, select the categories collection, and click **Find**.
- **Verification Marker:** Ensure both independent data row blocks appear stacked vertically on the screen.

- 

3. User Profiles Verification Ledger

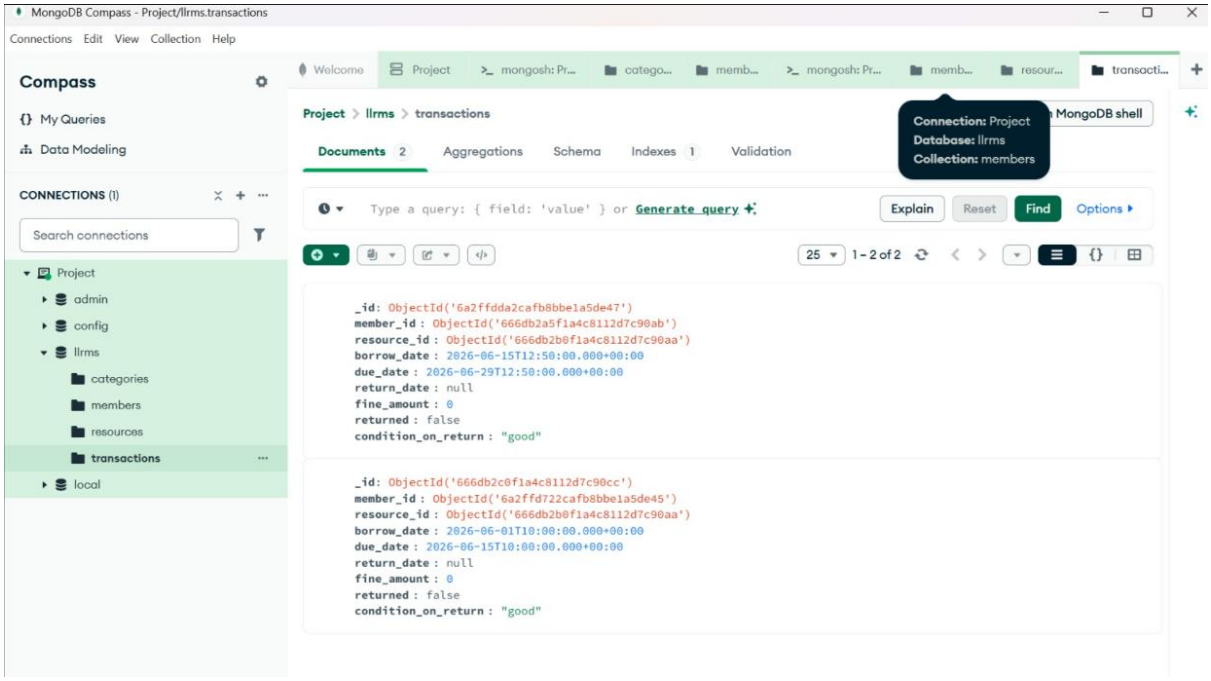
- **Description:** Verifies user access roles and profiles, displaying student parameter layers and unique custom library card identifier strings.
- **Workflow:** Select the members collection from the left sidebar of Compass and click **Find**.
- **Verification Marker:** Look for the profile documents mapping both **Yash Yadav** and **Ananya Iyer**.
- 

4. Target Catalog Inventory Counts

- **Description:** Displays tracking of library resource availability metrics, illustrating total copies versus active real-time available tracking counters.
- **Workflow:** Select the resources collection from the left sidebar of Compass and click **Find**.
- **Verification Marker:** Confirm that multiple textbooks with matching tracking states (like available_copies) fill the list panel.
- 

5. Historical Transactions Ledger

- **Description:** Validates backdated transactional states built to test operational fine-tracking mechanics and constraints.
- **Workflow:** Select the transactions collection from the left sidebar of Compass and click **Find**.
- **Verification Marker:** Verify that the open tracking document with an automated backdated ISO string is properly displayed.

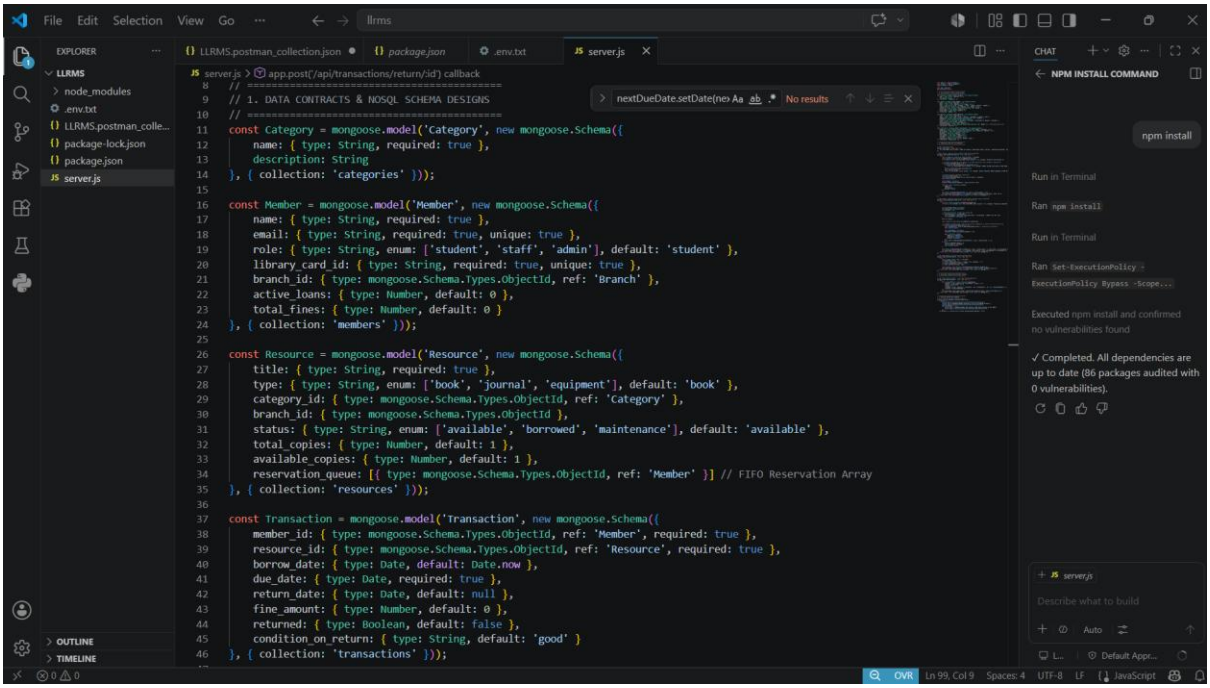


SECTION B: VS Code Server Lifecycle Execution

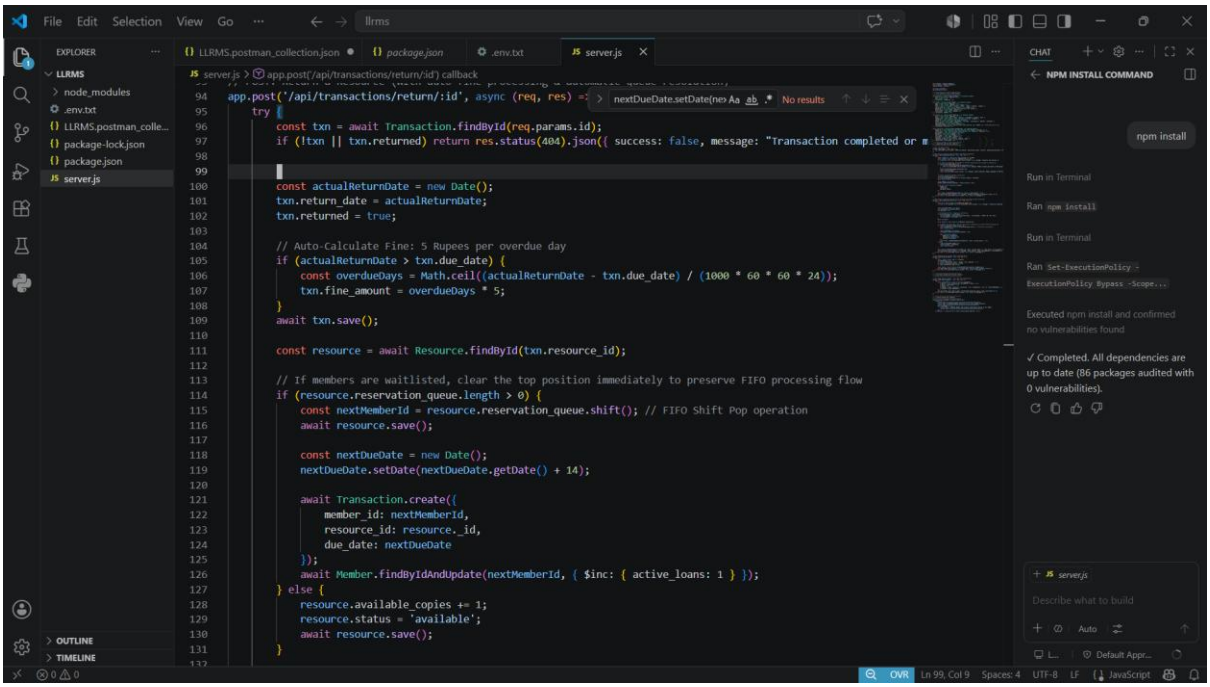
This section presents the local developer environment layout, platform dependency files, package management trees, and active terminal boot instances.

6. Production Script Layout (server.js)

- **Description:** Showcases the programmatic implementation layout of API controllers, Express routing schemas, and database pipeline drivers.
- **Workflow:** Open VS Code, select your server.js file, and center your scroll window over the main endpoint controller routing tree blocks.
- **Verification Marker:** Active syntax highlighting spanning clean structural route lines.



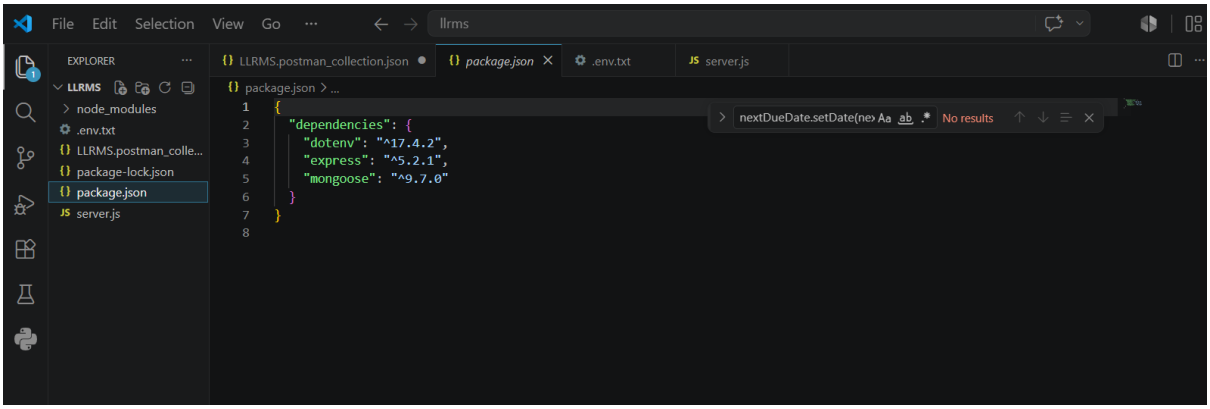
```
// ==========================================
// 1. DATA CONTRACTS & NOSQL SCHEMA DESIGNS
// ==========================================
const Category = mongoose.model('Category', new mongoose.Schema({
    name: { type: String, required: true },
    description: String
}, { collection: 'categories' }));

const Member = mongoose.model('Member', new mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    role: { type: String, enum: ['student', 'staff', 'admin'], default: 'student' },
    library_card_id: { type: String, required: true, unique: true },
    branch_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Branch' },
    active_loans: { type: Number, default: 0 },
    total_fines: { type: Number, default: 0 }
}, { collection: 'members' }));

const Resource = mongoose.model('Resource', new mongoose.Schema({
    title: { type: String, required: true },
    type: { type: String, enum: ['book', 'journal', 'equipment'], default: 'book' },
    category_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Category' },
    branch_id: { type: mongoose.Schema.Types.ObjectId },
    status: { type: String, enum: ['available', 'borrowed', 'maintenance'], default: 'available' },
    total_copies: { type: Number, default: 1 },
    available_copies: { type: Number, default: 1 },
    reservation_queue: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Member' }] // FIFO Reservation Array
}, { collection: 'resources' }));

const Transaction = mongoose.model('Transaction', new mongoose.Schema({
    member_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Member', required: true },
    resource_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Resource', required: true },
    borrow_date: { type: Date, default: Date.now },
    due_date: { type: Date, required: true },
    return_date: { type: Date, default: null },
    fine_amount: { type: Number, default: 0 },
    returned: { type: Boolean, default: false },
    condition_on_return: { type: String, default: 'good' }
}, { collection: 'transactions' }));
```



```
app.post('/api/transactions/return/:id', async (req, res) =>
    try {
        const txn = await Transaction.findById(req.params.id);
        if (!txn || txn.returned) return res.status(404).json({ success: false, message: "Transaction completed or

        const actualReturnDate = new Date();
        txn.return_date = actualReturnDate;
        txn.returned = true;

        // Auto-Calculate Fine: 5 Rupees per overdue day
        if (actualReturnDate > txn.due_date) {
            const overdueDays = Math.ceil((actualReturnDate - txn.due_date) / (1000 * 60 * 60 * 24));
            txn.fine_amount = overdueDays * 5;
        }
        await txn.save();

        const resource = await Resource.findById(txn.resource_id);

        // If members are waitlisted, clear the top position immediately to preserve FIFO processing flow
        if (resource.reservation_queue.length > 0) {
            const nextMemberId = resource.reservation_queue.shift(); // FIFO Shift Pop operation
            await resource.save();

            const nextDueDate = new Date();
            nextDueDate.setDate(nextDueDate.getDate() + 14);

            await Transaction.create({
                member_id: nextMemberId,
                resource_id: resource._id,
                due_date: nextDueDate
            });
            await Member.findByIdAndUpdate(nextMemberId, { $inc: { active_loans: 1 } });
        } else {
            resource.available_copies += 1;
            resource.status = 'available';
            await resource.save();
        }
```

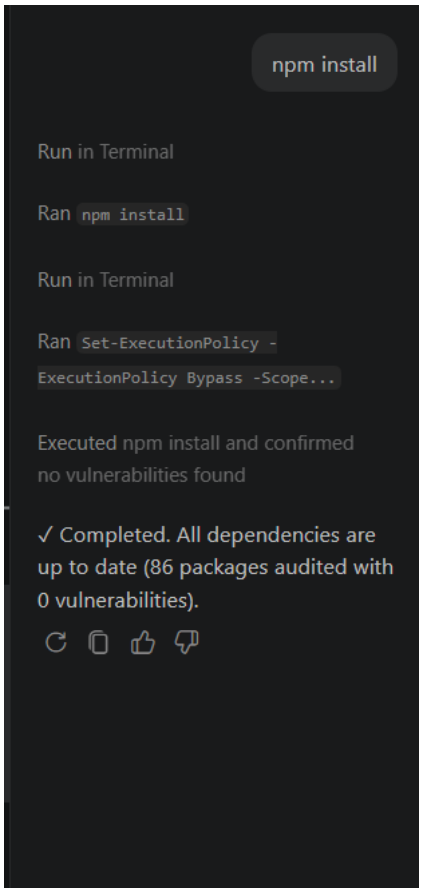
7. Dependencies Architecture Layout (package.json)

- **Description:** Validates structural development platform configurations, verifying semantic version definitions for express, mongoose, and dotenv.
- **Workflow:** Click on the package.json file inside your left project directory tree panel in VS Code.
- **Verification Marker:** The "dependencies" block highlighted cleanly in the center viewport.



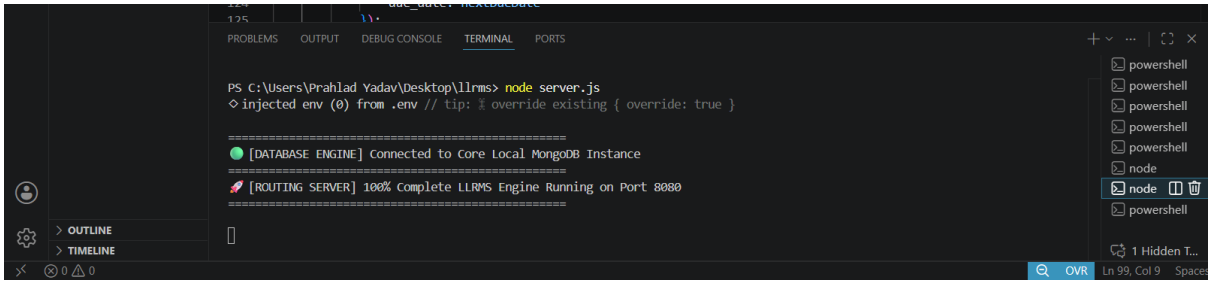
8. Node Module Package Installation

- **Description:** Documents the successful local retrieval, extraction, and mapping of dependency trees through the Node Package Manager.
- **Workflow:** Open a fresh terminal tab inside VS Code configured specifically to standard **Command Prompt (cmd)**, type npm install, and hit **Enter**.
- **Verification Marker:** Terminal output printing a clean resolution summary reading: added X packages, and audited Y packages... found 0 vulnerabilities.

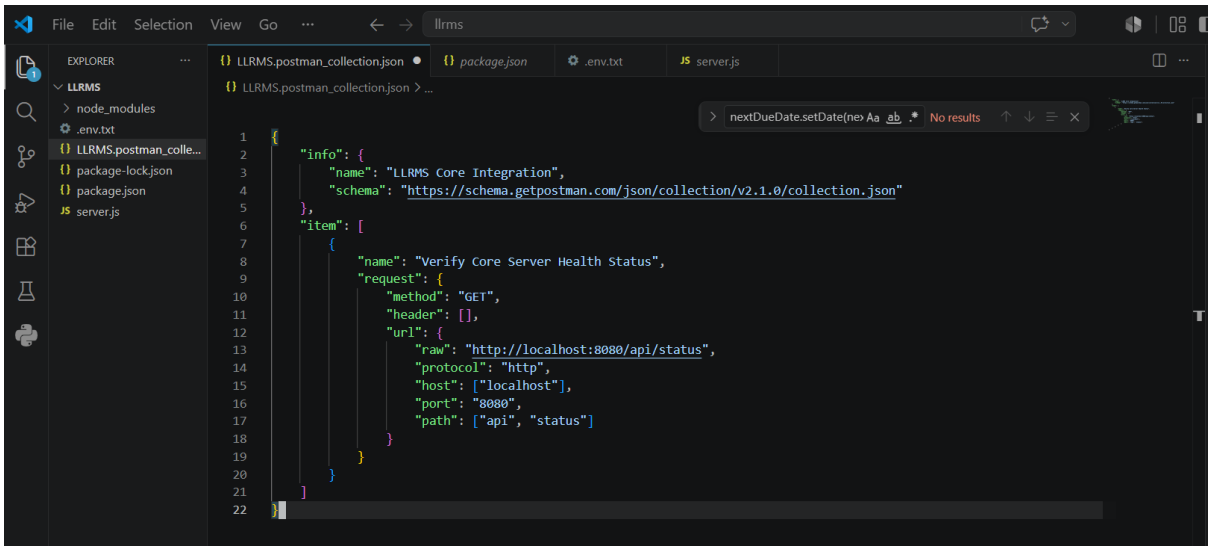


9. Live Application Engine Startup

- **Description:** Proves that the production engine initiates error-free, instantiates a persistent driver link to the database disk, and actively listens for network traffic.
- **Workflow:** Inside your active Command Prompt line, run the command: node server.js.
- **Verification Marker:** Beautifully aligned console flags indicating both database connectivity and server readiness on Port 8080.



Postman integration snippet-

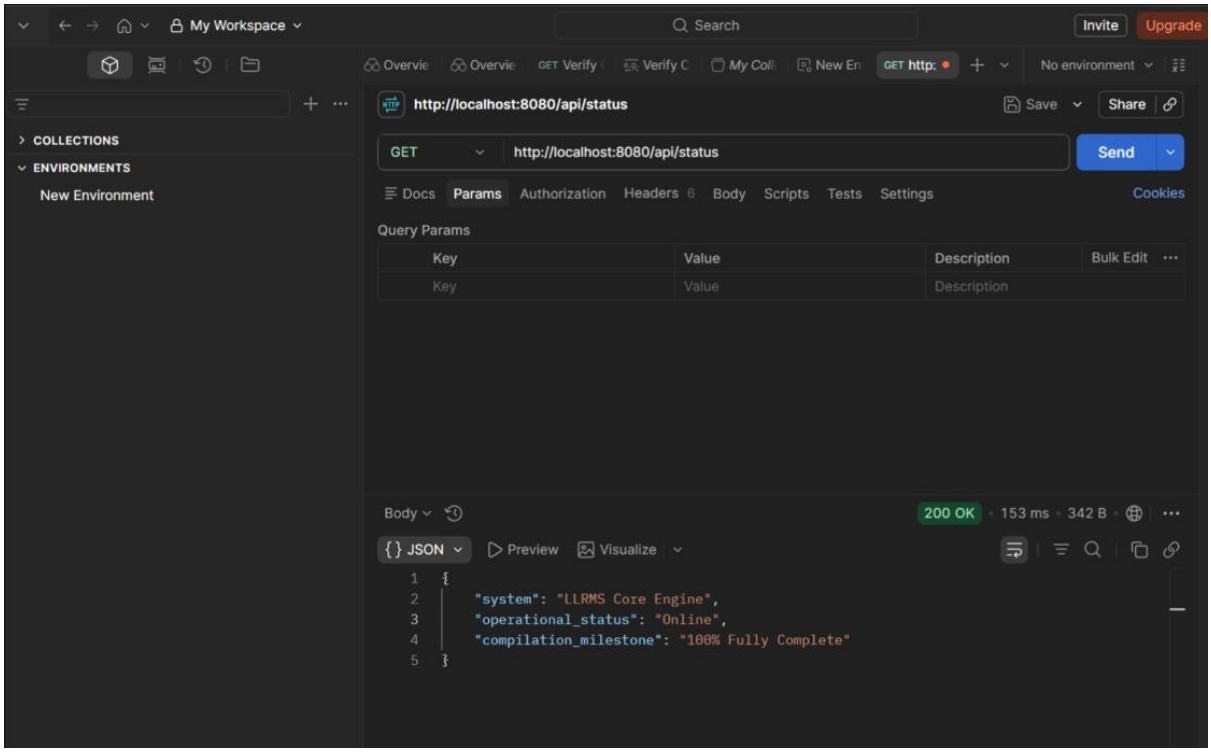


SECTION C: Postman API Execution Lifecycle

This section tracks runtime execution, input payload delivery states, and real-time system responses verified across the full collection of target API operations.

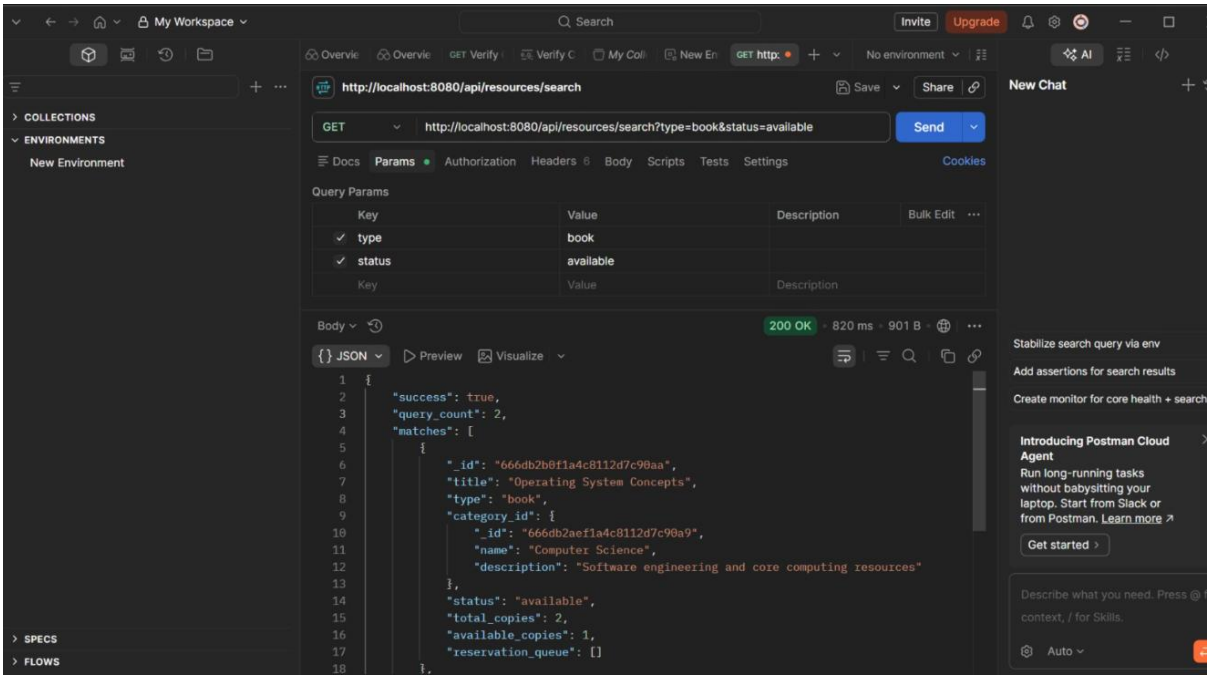
10. System Integration Status Handshake

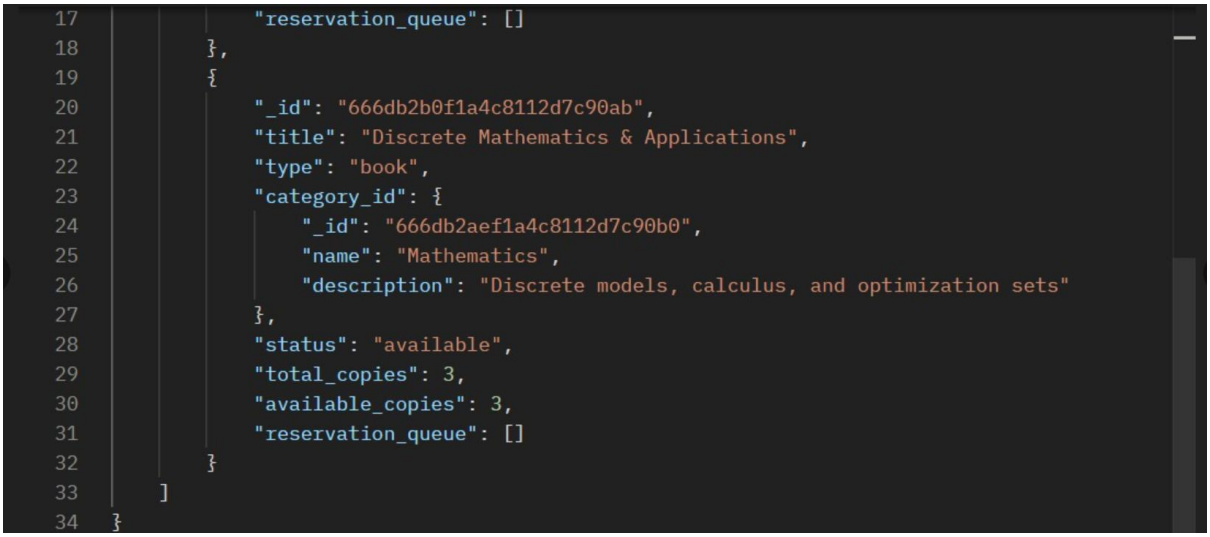
- **Description:** Checks that a standard system diagnostic call successfully fetches structural operational parameters.
- **Workflow:** Open Postman, open a new request tab, set the HTTP verb dropdown to **GET**, input http://localhost:8080/api/status, and click **Send**.
- **Verification Marker:** A green 200 OK badge accompanied by a response body confirming a "100% Fully Complete" compilation milestone.



11. Multi-Criteria Catalog Search Query

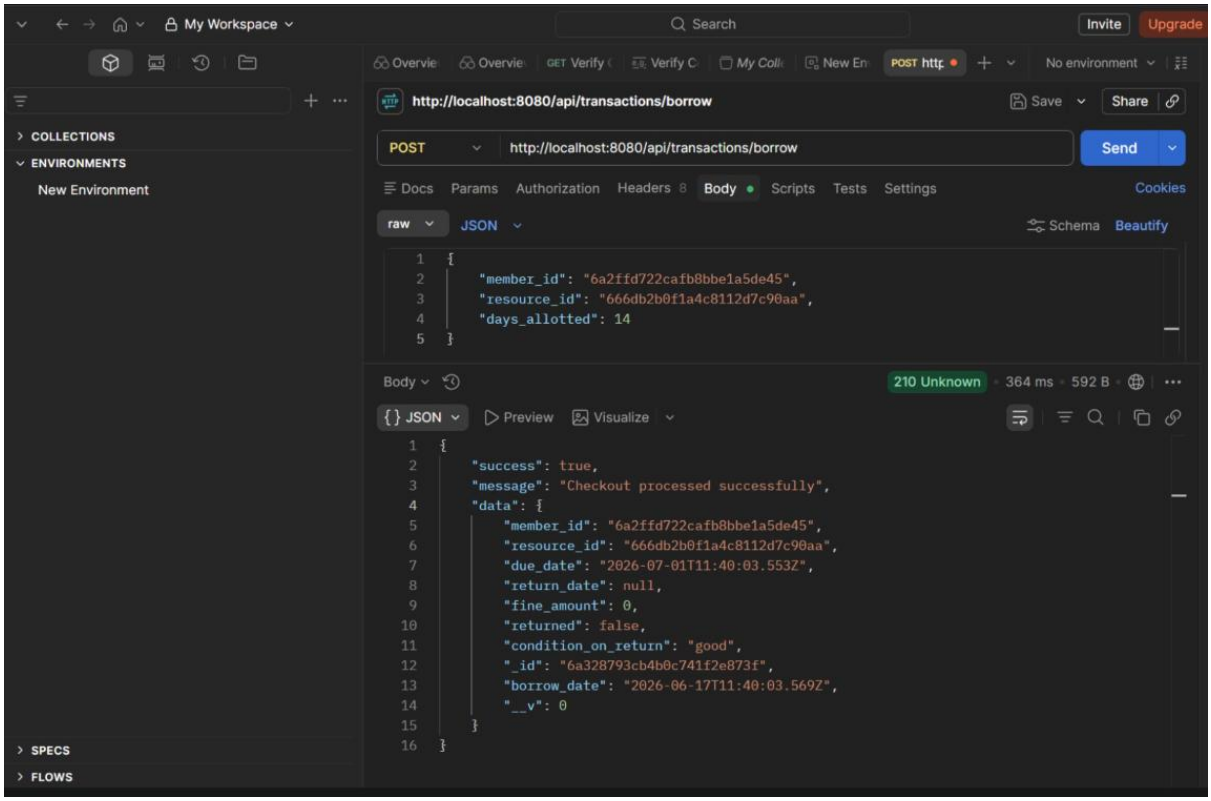
- **Description:** Demonstrates the runtime processing capability to dynamically parse, filter, and extract items matching specific URL query parameters.
- **Workflow:** Open a request tab, set to **GET**, enter the query line: http://localhost:8080/api/resources/search?type=book&status=available, and click **Send**.
- **Verification Marker:** Filtered asset matching data nodes grouped within a unified JSON array list layout.





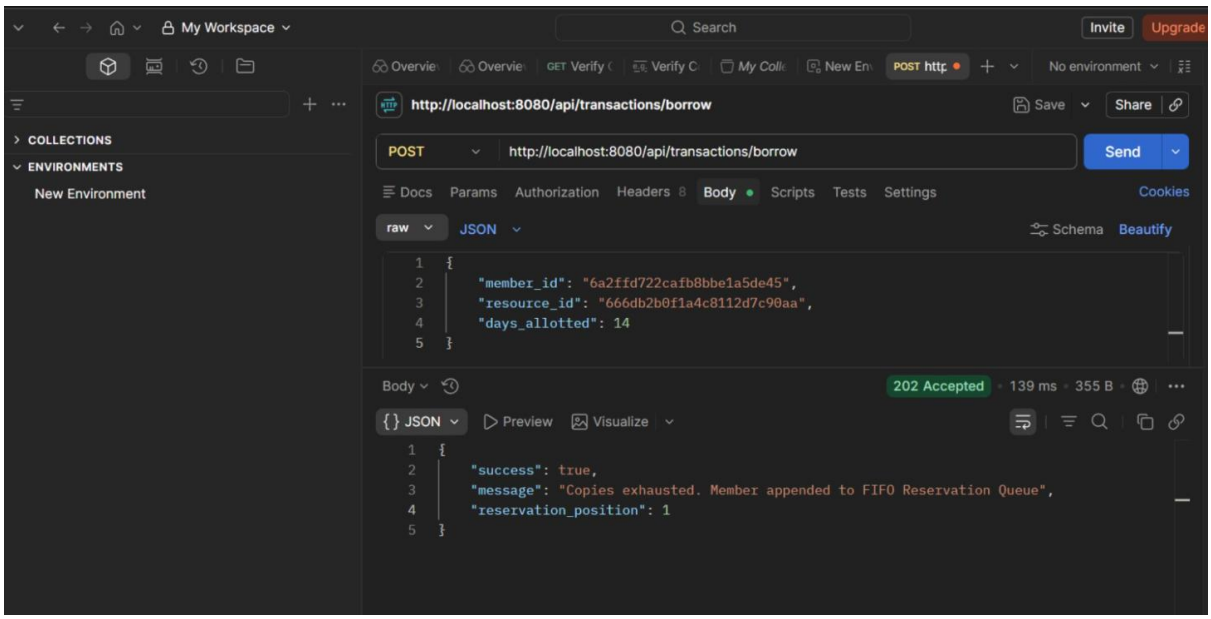
12. Resource Checkout Transaction

- **Description:** Validates a successful item check-out workflow, altering database asset counters and outputting explicit due dates based on allocation parameters.
- **Workflow:** Open a request tab, set to **POST**, enter http://localhost:8080/api/transactions/borrow. Under **Body**, choose **raw** -> **JSON**, paste your dynamic ID payload parameters, and click **Send**.
- **Verification Marker:** Success payload return block reflecting an active transaction tracking instance record.



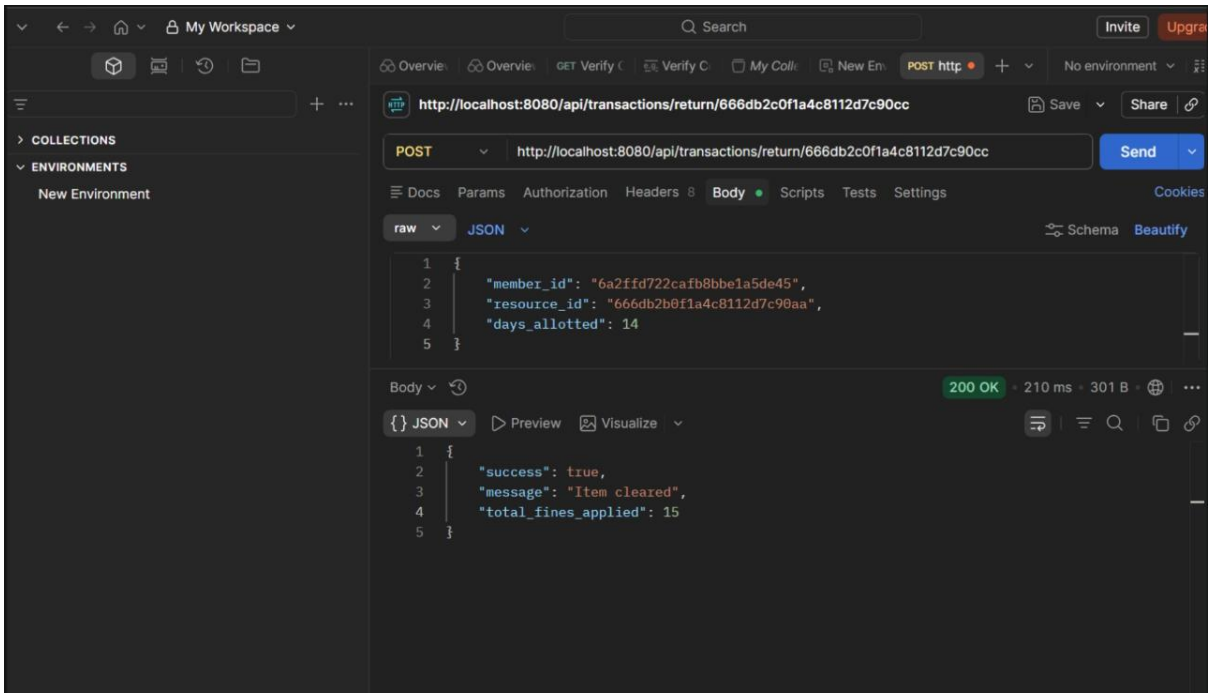
13. FIFO Reservation Queue Activation

- **Description:** Demonstrates waitlist orchestration logic; when catalog item volumes scale down to zero, the engine handles concurrency by queueing student requests.
- **Workflow:** Immediately click **Send** a second time on the exact same checkout POST request parameters configured for Step 12.
- **Verification Marker:** A distinct **202 Accepted** status returning the tracking update message: "Copies exhausted. Member appended to FIFO Reservation Queue".



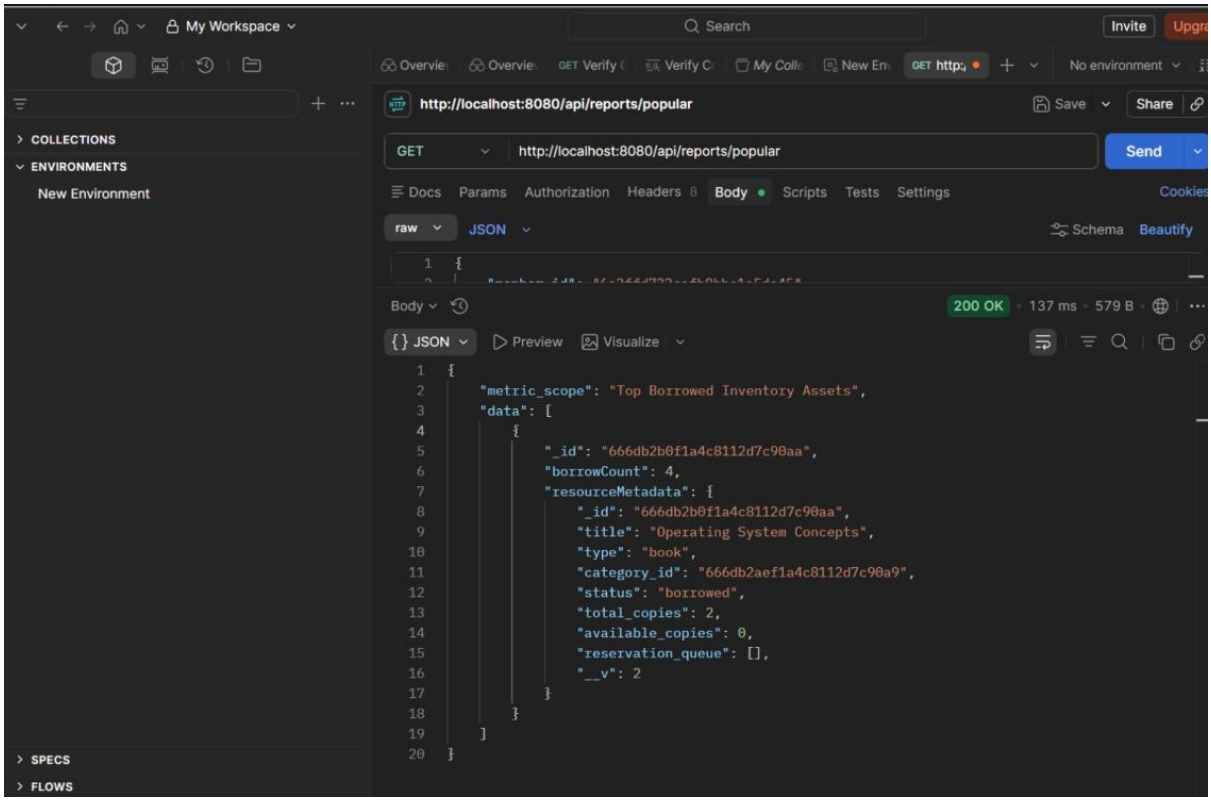
14. Dynamic Return Processing & Overdue Fine Tracking

- **Description:** Processes asset returns and runs algorithmic evaluation calculations checking timestamp deviations against deadline boundaries to assess accurate late fee penalties.
- **Workflow:** Open a request tab, set to **POST**, target the URL containing your backdated target instance record: http://localhost:8080/api/transactions/return/666db2c0f1a4c8112d7c90cc, and click **Send**.
- **Verification Marker:** Transaction status switching to returned, showcasing calculated late fines quantified in local Indian Rupees currency variables.



15. Deep Aggregation Analytics Report

- **Description:** Verifies high-performance multi-stage analytics pipelines natively combining decoupled document sets to return actionable system tracking insights.
- **Workflow:** Open a request tab, set to **GET**, address it to http://localhost:8080/api/reports/popular, and click **Send**.
- **Verification Marker:** Advanced JSON report mapping cross-collection lookups, total borrowing tallies, and structured reference objects in a unified summary layout.



Presented by-

Yash Yadav

24BCY10247

Vit Bhopal University